Lista de Exercícios 4 - Manipulação de Objetos com C#

Prof. Dr. Rodrigo Colnago Contreras
contreras@unifesp.br
Lattes Google Scholar ORCID

Criada em 21/10/2025 e modificada em 21/04/2026.

Exercícios

Exercício 1. Considere o trecho de código abaixo, escrito em C#:

```
internal class A
    {
        private int x;
        protected int y;
        public int z;

        public A(int x, int y, int z)
        {
            this.x = x;
            this.y = y;
            this.z = z;
        }
        public A(int x, int y, int z)
        {
            this.x = x + 1;
            this.y = y + 2;
            this.z = z + 3;
        }
    }
```

- (a) Explique qual é o papel do identificador `this` neste código e por que ele é necessário no contexto apresentado.
- (b) O interpretador do C# será capaz de definir a classe `A` sem exceções ou erros? Caso negativo, diga qual conceito de orientação a objetos não foi respeitado e sugira uma maneira de corrigir o problema do código.

Exercício 2.
Considere a classe `B`, herdeira da classe `A` definida no Exercício 1 anterior e corrigida para não apresentar mais erros, definida na sequência:

```
internal class B : A
{
    private int w;

    public B(int w)
    {
        this.w = w;

        MostraValores();
    }

    public void MostraValores()
    {
        Console.WriteLine($"x = {this.x}, y = {this.y}, z = {this.z}, w
            = {this.w}");
    }
}
```

Sabe-se que existem dois problemas relacionados à orientação a objetos em C# com a definição de `B`. Destaque-os e comente como corrigi-los.

Exercício 3. Considere a classe `Q` definida a seguir:

```
 internal class Q
{
    private int Numerador;
    private int Denominador;

    public Q(int numerador, int denominador)
    {
        SetNumerador(numerador);
        SetDenominador(denominador);
    }

    public void SetNumerador(int n)
    {
        Numerador = n;
    }
    public int GetNumerador()
    {
        return Numerador;
    }

    public void SetDenominador(int n)
    {
        if(n != 0)
        {
            Denominador = n;
        }
        else
        {
            throw new Exception("Nao eh possivel definir denominador
                nulo.");
        }
```

```
        }
        public int GetDenominador()
        {
            return Denominador;
        }
    }
```

A função `Main` do programa principal possui o seguinte conteúdo:

```
    static void Main(string[] args)
        {
            Q   q1 = new Q(1,2),
                q2 = new Q(1,1),
                q3,
                q4 = new Q(1,0);

            Console.WriteLine(q1);

            Console.WriteLine(q1 + q2);

            Console.ReadKey();
        }
```

Neste caso, disserte sobre a veracidade das seguintes afirmações:

(a) O primeiro `Console.WriteLine` apresentará o valor `0.5` na tela.

(b) O segundo `Console.WriteLine` apresentará o valor `1.5` na tela.

(c) Uma forma de apresentar o valor de `q1` de maneira fracionária na tela seria por meio do seguinte comando de escrita: `Console.WriteLine($"{q1.Numerador} / {q1.Denominador}");`.

(d) `q3` é um objeto do tipo `Q`.

(e) O código do método `Main` não será executado corretamente por duas razões. Primeiro, o segundo comando de saída não conseguirá processar a expressão `q1 + q2`, pois a classe `Q` não define uma sobrecarga para o operador `+`. Segundo, a criação do objeto `q4` resultará em uma exceção, uma vez que, ao atribuir o valor 0 ao atributo `Denominador`, é acionada a verificação que lança uma exceção para impedir uma atribuição desse tipo para tal atributo.

Exercício 4. Considere o código a seguir, que utiliza a classe `Q` definida no exercício anterior, agora aplicada em um vetor de objetos:

```
internal class Program
{
    static void Main(string[] args)
    {
        Q[] fracoes = new Q[3];

        fracoes[0].SetNumerador(1);
        fracoes[0].SetDenominador(2);

        fracoes[1] = new Q(3, 4);

        fracoes[2].SetNumerador(5);
```

```
13          fracoes[2].SetDenominador(6);
14
15          for (int i = 0; i < fracoes.Length; i++)
16          {
17              Console.WriteLine($"Fracao {i+1}: {fracoes[i].GetNumerador()}/{
                    fracoes[i].GetDenominador()}");
18          }
19
20          Console.ReadKey();
21      }
22  }
```

Analise o código e responda:

(a) O código acima "compila" e executa corretamente? Justifique sua resposta.

(b) Explique o que de fato é criado em memória quando o comando `Q[] fracoes = new Q[3];` é executado.

(c) Por que ocorre um erro ao executar as linhas `fracoes[0].SetNumerador(1);` e `fracoes[2].SetDenominador(6);`?

(d) Reescreva o trecho de código necessário para que o vetor de objetos `fracoes` seja corretamente instanciado e o programa execute sem erros.

(e) Explique a diferença conceitual entre *definir o tamanho de um vetor* e *instanciar os objetos que o mesmo armazena*.